

Part IV: Training Dynamics of Neural Networks

RTG Summer School

Yimin Zhong yimin.zhong@auburn.edu

Random Features & NNGP

Fundamental questions about neural network training:

1. **Convergence:** Why does gradient descent find a global minimum despite non-convexity?
2. **Generalization:** Why do overparameterized networks (more params than data) not overfit?
3. **Inductive bias:** What functions does the architecture + optimizer combination prefer?

Infinite Width: As the number of hidden neurons $m \rightarrow \infty$, the randomness of initialization averages out. The network becomes a *deterministic* kernel operator. Infinite width is a standard setting for studying overparameterization.

At initialization. The network's weights are random. The NNGP (Neural Network Gaussian Process) framework tells us what the untrained network looks like.

A Network at Initialization

Consider a 2-layer network with m hidden neurons:

$$f(\mathbf{x}; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(\mathbf{w}_i^T \mathbf{x}), \quad \mathbf{w}_i \sim \mathcal{N}(\mathbf{0}, I_d), \quad a_i \sim \mathcal{N}(0, 1)$$

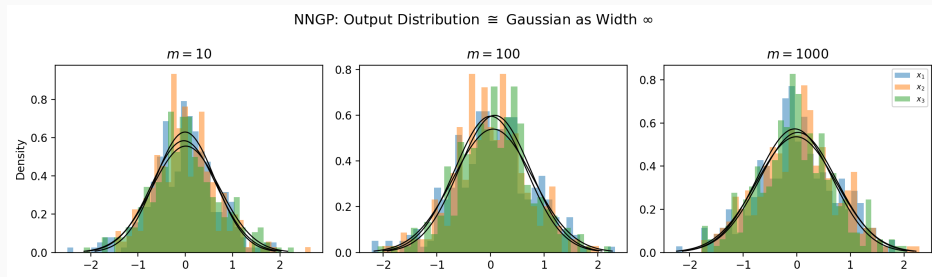
The $1/\sqrt{m}$ scaling keeps the output variance $O(1)$ as m grows.

Key observation: For a fixed input $\mathbf{x}$, the pre-activations $\{\mathbf{w}_i^T \mathbf{x}\}$ are i.i.d. Gaussians with variance $\|\mathbf{x}\|^2$. The output $f(\mathbf{x})$ is a sum of m i.i.d. terms — by the **Central Limit Theorem**:

$$f(\mathbf{x}) \xrightarrow{m \rightarrow \infty} \mathcal{N}(0, K(\mathbf{x}, \mathbf{x}))$$

At infinite width, the network at initialization is a **Gaussian process**.

NNGP Convergence



1-hidden-layer ReLU net, 300 random initializations per width. At $m = 10$, distributions are irregular. At $m = 1000$, they're indistinguishable from Gaussians (black curves). The NNGP limit kicks in at practical widths.

Neal's Theorem: The NNGP Correspondence

Theorem (Neal, 1994)

As $m \rightarrow \infty$, a 1-hidden-layer network with Gaussian priors converges in distribution to a Gaussian process:

$$f(\cdot) \xrightarrow{d} \mathcal{GP}(\mathbf{0}, K), \quad K(\mathbf{x}, \mathbf{x}') = \mathbb{E}_{\mathbf{w} \sim \mathcal{N}(\mathbf{0}, I)} [\sigma(\mathbf{w}^T \mathbf{x}) \sigma(\mathbf{w}^T \mathbf{x}')]]$$

The kernel K is called the **NNGP kernel** — deterministic, determined by architecture and activation alone.

The architecture is the prior. Different activation functions induce different covariance structures, encoding different inductive biases before any data is seen.

Explicit NNGP Kernels for Common Activations

On $\|x\| = \|x'\| = 1$, let $u = x^T x'$ and $\theta = \arccos(u)$:

Activation	$\sigma(z)$	$K(x, x')$
Linear	z	u
ReLU	$\max(0, z)$	$\frac{1}{2\pi} (\sin \theta + (\pi - \theta) \cos \theta)$
Erf	$\text{erf}(z)$	$\frac{2}{\pi} \arcsin\left(\frac{2u}{3}\right)$
Step	$\mathbf{1}_{z>0}$	$\frac{1}{2} - \frac{\theta}{2\pi}$
GELU	$z\Phi(z)$	(no simple closed form)

All of these are **rotation-invariant kernels** on the sphere — they depend only on $\langle x, x' \rangle$. This is the signature of a fully-connected architecture with Gaussian weights. Sources: Neal (1994), Williams (1997), Cho & Saul (2009).

Deep NNGP: Recursive Kernel Composition

For an L -layer network, kernels compose layer by layer. Start with $K^{(0)}(\mathbf{x}, \mathbf{x}') = \frac{\sigma_w^2}{d} \mathbf{x}^T \mathbf{x}' + \sigma_b^2$.

For $\ell = 1, \dots, L$, pre-activations are jointly Gaussian:

$$\begin{pmatrix} h_i^{(\ell)}(\mathbf{x}) \\ h_i^{(\ell)}(\mathbf{x}') \end{pmatrix} \sim \mathcal{N}(\mathbf{0}, \Sigma_\ell), \quad \Sigma_\ell = \begin{pmatrix} K^{(\ell-1)}(\mathbf{x}, \mathbf{x}) & K^{(\ell-1)}(\mathbf{x}, \mathbf{x}') \\ K^{(\ell-1)}(\mathbf{x}', \mathbf{x}) & K^{(\ell-1)}(\mathbf{x}', \mathbf{x}') \end{pmatrix}$$

Kernel update by integrating against this 2D Gaussian:

$$K^{(\ell)}(\mathbf{x}, \mathbf{x}') = \sigma_w^2 \mathbb{E}_{(\mathbf{z}, \mathbf{z}') \sim \mathcal{N}(\mathbf{0}, \Sigma_\ell)} [\sigma(\mathbf{z}) \sigma(\mathbf{z}')] + \sigma_b^2$$

For ReLU: closed-form **arc-cosine kernel**: $K^{(\ell)}(\mathbf{x}, \mathbf{x}') = \frac{1}{2\pi} \sqrt{K^{(\ell-1)}(\mathbf{x}, \mathbf{x}) K^{(\ell-1)}(\mathbf{x}', \mathbf{x}')} \cdot J_1(\theta_\ell)$,
where $J_1(\theta) = \sin \theta + (\pi - \theta) \cos \theta$ and $\theta_\ell = \arccos(K^{(\ell-1)}(\mathbf{x}, \mathbf{x}') / \sqrt{K^{(\ell-1)}(\mathbf{x}, \mathbf{x}) K^{(\ell-1)}(\mathbf{x}', \mathbf{x}')})$.

Fixed Point of the Deep NNGP Recursion

As $\ell \rightarrow \infty$, the kernel converges to a fixed point $K^*(x, x')$ that satisfies:

$$K^*(x, x') = \sigma_w^2 \mathbb{E}_{(z, z') \sim \mathcal{N}(0, \Sigma_*)} [\sigma(z) \sigma(z')] + \sigma_b^2$$

Key insight (Schoenholz et al., 2017): The convergence behavior depends on σ_w^2 :

- $\sigma_w^2 < 1$: **Ordered phase.** K^* degenerates to a constant — all inputs look identical. Information about x is lost.
- $\sigma_w^2 = 1$: **Critical line.** K^* is non-degenerate. Signals propagate stably.
- $\sigma_w^2 > 1$: **Chaotic phase.** Gradients explode/ vanish. Training becomes impossible.

The **Edge of Chaos** ($\sigma_w^2 = 1$) is where deep networks are trainable. This is the initialization that modern architectures use.

Random Feature Interpretation

For finite m , view the network as a **linear model** over random features:

$$f(x) = \sum_{i=1}^m a_i \cdot \underbrace{\frac{1}{\sqrt{m}} \sigma(w_i^T x)}_{\phi_i(x)}, \quad \{w_i\} \text{ frozen}$$

If hidden weights stay fixed, training = kernel ridge regression with the NNGP kernel.

Key question

Do the weights actually move in this setting?

Neural Tangent Kernel

Gradient Flow: The Continuous-Time Limit

Before analyzing the NTK, recall **gradient flow** — the $\eta \rightarrow 0$ limit of gradient descent:

$$\dot{\theta}_t = -\nabla_{\theta} \mathcal{L}(\theta_t)$$

For MSE loss $\mathcal{L}(\theta) = \frac{1}{2} \sum_{i=1}^N (f(x_i; \theta) - y_i)^2$:

$$\dot{\theta}_t = -\sum_{i=1}^N \nabla_{\theta} f(x_i; \theta_t) \cdot (f(x_i; \theta_t) - y_i)$$

Gradient Flow is easier in analysis

- Removes discretization artifacts.
- For small enough η , GD closely tracks gradient flow on $O(1/\eta)$ timescales.
- The ODE formulation allows us to apply spectral theory to analyze convergence.

The output on any test point x evolves via the chain rule:

$$\dot{f}(x) = \langle \nabla_{\theta} f(x; \theta_t), \dot{\theta}_t \rangle = - \sum_{i=1}^N \underbrace{\langle \nabla_{\theta} f(x), \nabla_{\theta} f(x_i) \rangle}_{\Theta_t(x, x_i)} \cdot (f(x_i) - y_i)$$

Definition (Neural Tangent Kernel)

$\Theta_t(x, x') = \langle \nabla_{\theta} f(\theta_t, x), \nabla_{\theta} f(\theta_t, x') \rangle$ is a symmetric kernel.

For instance, if $f(x) = \sum_{i=1}^m a_i \sigma(w_i x)$, the parameters $\theta = (a_i, w_i)_{1 \leq i \leq m}$,

$$\begin{aligned} \nabla_{\theta} f(x) &= (\nabla_{a_i} f(x), \nabla_{w_i} f(x))_{1 \leq i \leq m} \\ &= (\sigma(w_i x), a_i x \sigma'(w_i x))_{1 \leq i \leq m} \end{aligned}$$

This **inner product of gradients** governs how an update at x' affects the prediction at x .

In vector form over the training set $X = \{x_1, \dots, x_N\}$ (discrete data set):

$$\dot{f}(X) = -\Theta_t(X, X) \cdot (f(X) - Y)$$

If Θ_t is constant, this is a linear ODE with closed-form solution.

Since the weights change over time, the NTK Θ_t is also time-dependent.

The NTK Regime: Infinite Width Freezes the Kernel

Theorem (Jacot, Gabriel, Hongler, 2018)

Under NTK parameterization ($1/\sqrt{m}$ output scaling), as $m \rightarrow \infty$: $\Theta_t(x, x') \rightarrow \Theta_\infty(x, x')$ for all $t \geq 0$.
The kernel is **constant**.

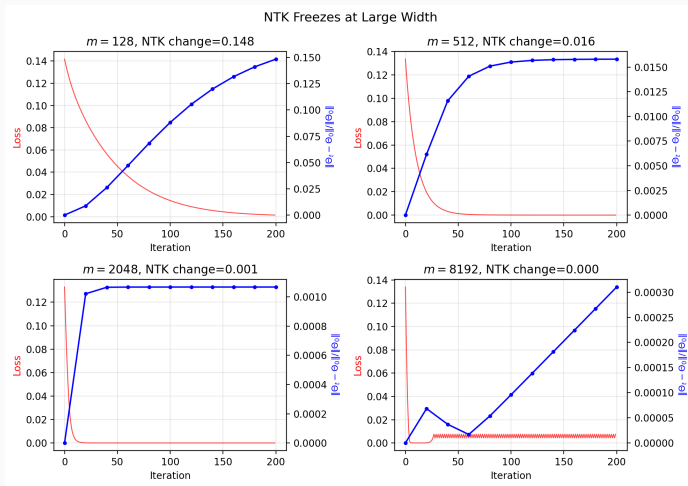
Proof sketch: The change in the kernel is bounded by the Hessian:

$$\|\Theta_t - \Theta_0\| \leq \|\nabla_{\theta}^2 f\| \cdot \|\theta_t - \theta_0\|$$

- $\|\nabla_{\theta}^2 f\| = O(1)$: each parameter's Hessian is bounded. The activation function needs to be $\mathcal{C}^2$.
- $\|\theta_t - \theta_0\| = O(1/\sqrt{m})$: parameters barely need to move because output scaling $1/\sqrt{m}$ means tiny weight changes produce $O(1)$ output changes.

Consequence: Infinite-width training = **kernel regression** with fixed kernel. Parameters barely move, yet the function interpolates exactly. The network is *lazy* and does not learn.

NTK training



Left: training loss by width. Right: relative NTK change $\|\Theta_t - \Theta_0\|/\|\Theta_0\|$.

Training Dynamics Under the NTK

With constant Θ_∞ , the ODE is linear and time-invariant:

$$\dot{f}_t(X) = -\Theta_\infty \cdot (f_t(X) - Y)$$

Closed-form solution via matrix exponential:

$$f_t(X) = (I - e^{-\Theta_\infty t})Y + e^{-\Theta_\infty t}f_0(X)$$

If $f_0(X) \approx 0$ (small initialization): $f_t(X) \approx (I - e^{-\Theta_\infty t})Y$.

If $\Theta_\infty \succ 0$, then $e^{-\Theta_\infty t} \rightarrow 0$ as $t \rightarrow \infty$, so $f_t(X) \rightarrow Y$ **exponentially fast**.

Convergence per eigendirection: Diagonalize $\Theta_\infty = \sum_{k=1}^N \lambda_k v_k v_k^T$. Project error:

$e_k(t) = v_k^T(f_t(X) - Y)$. Then:

$$\dot{e}_k = -\lambda_k e_k \quad \implies \quad e_k(t) = e_k(0) \cdot e^{-\lambda_k t}$$

Large eigenvalues $\rightarrow$ fast convergence. Small eigenvalues $\rightarrow$ slow convergence. This asymmetry is the **spectral bias**.

NTK for a 2-Layer ReLU Network

For $f(x) = \frac{1}{\sqrt{m}} \sum a_i \sigma(w_i^T x)$, the gradient splits into two parts:

$$\nabla_{w_i} f = \frac{a_i}{\sqrt{m}} \sigma'(w_i^T x) x, \quad \nabla_{a_i} f = \frac{1}{\sqrt{m}} \sigma(w_i^T x)$$

The NTK becomes:

$$\Theta_{\infty} = \underbrace{\mathbb{E}_w[\sigma(w^T x) \sigma(w^T x')]}_{\text{NNGP}} + (x^T x') \underbrace{\mathbb{E}_w[\sigma'(w^T x) \sigma'(w^T x')]}_{\text{correction}}$$

For $\|x\| = \|x'\| = 1, \theta = \arccos(x^T x')$:

$$\Theta_{\infty}(x, x') = \frac{1}{2\pi} (\sin \theta + (\pi - \theta) \cos \theta) + \frac{1}{2\pi} (\pi - \theta) x^T x'$$

The NTK = NNGP kernel + a **first-order correction** from the derivative of the activation. Both terms depend only on the angle between inputs and the kernel is rotation-invariant on $\mathbb{S}^{d-1}$.

NTK for Deep Networks: Recursive Formula

For an L -layer fully-connected network, the NTK also composes recursively. Define:

$$\Sigma^{(\ell)}(x, x') = \begin{pmatrix} K^{(\ell-1)}(x, x) & K^{(\ell-1)}(x, x') \\ K^{(\ell-1)}(x', x) & K^{(\ell-1)}(x', x') \end{pmatrix}$$
$$\dot{\Sigma}^{(\ell)}(x, x') = \begin{pmatrix} \Theta^{(\ell-1)}(x, x) & \Theta^{(\ell-1)}(x, x') \\ \Theta^{(\ell-1)}(x', x) & \Theta^{(\ell-1)}(x', x') \end{pmatrix}$$

Then the NTK at layer ℓ is:

$$\Theta^{(\ell)}(x, x') = K^{(\ell)}(x, x') + \sigma_w^2 \Theta^{(\ell-1)}(x, x') \mathbb{E}_{(z, z') \sim \mathcal{N}(0, \Sigma^{(\ell)})}[\sigma'(z) \sigma'(z')]$$

Each layer adds a term: $\Theta^{(L)} = \sum_{\ell=1}^L K^{(\ell)} \cdot \prod_{k=\ell+1}^L \dot{K}^{(k)}$. The NTK is the **sum over all paths** through the network's gradient graph. Sources: Jacot et al. (2018), Lee et al. (2019).

NTK and Random Matrix Theory

When training data is random (e.g., $x_i \sim \text{Unif}(\mathbb{S}^{d-1})$), the NTK Gram matrix $\Theta_\infty(X, X) \in \mathbb{R}^{N \times N}$ is a **positive definite kernel matrix** as long as $x_i \neq x_j$, if $i \neq j$.

Key Consequences:

- The smallest eigenvalue $\lambda_{\min}(\Theta_\infty)$ controls the **convergence rate**: $\mathcal{L}(t) \sim e^{-2\lambda_{\min}t}$.
- For kernels with finite smoothness on $\mathbb{S}^{d-1}$, $\lambda_{\min} \rightarrow 0$ as $N \rightarrow \infty$, the Gram matrix becomes ill-conditioned once data are dense. This predicts slower training for higher eigenmodes (i.e., smaller eigenvalues).
- The rate of ill-conditioning depends on the **smoothness** of the activation: smoother activations (e.g., GELU) give faster eigenvalue decay.

Sources: Jacot et al. (2018), Bietti & Mairal (2019), Basri et al. (2019).

Spectral Bias

Spectral Bias: Low Frequencies First

Recall: error on eigendirection v_k decays as $e_k(t) = e_k(0) e^{-\lambda_k t}$.

Theorem (Spectral Bias for ReLU NTK on the Sphere (ArXiv:1912.01198))

Let $\mathbb{S}^{d-1} = \{x \in \mathbb{R}^d : \|x\| = 1\}$. The eigenfunctions of the ReLU NTK operator on $\mathbb{S}^{d-1}$ are the **spherical harmonics** $Y_{k,j}$ of degree $k = 0, 1, 2, \dots$.

The corresponding eigenvalues satisfy:

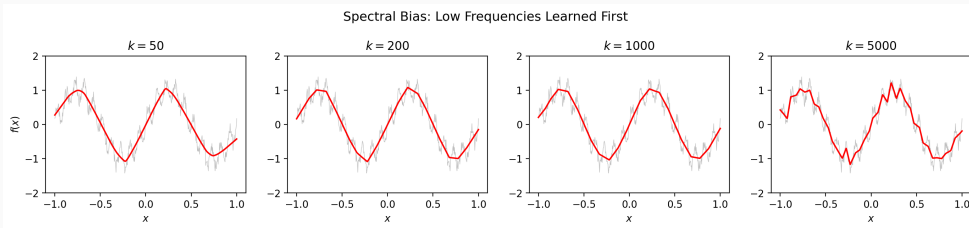
$$\lambda_k \sim k^{-d} \quad \text{as } k \rightarrow \infty$$

where k is the “frequency” (degree of the harmonic polynomial).

Smooth (low-degree) harmonics: large $\lambda_k \rightarrow$ learned exponentially fast. Oscillatory

(high-degree) harmonics: tiny $\lambda_k \rightarrow$ barely learned. The network is an **implicit low-pass filter**.

1D ReLU Spectral Bias



Target = $\sin(2\pi x)$ (low freq) + $0.3 \sin(20\pi x)$ (high freq). Step 50: only the low-frequency component is fit. Step 200: the high-frequency component begins to appear. Step 5000: both fully learned. The network learns the smooth structure first.

Generalization benefit: Real data has most energy at low frequencies. High frequencies are noise. The spectral bias automatically denoises.

The Frequency Principle (Xu et al., 2019): For general DNNs trained with SGD, the relative error of low-frequency components decreases strictly faster than high-frequency ones.

Beyond the NTK: Spectral Bias in Finite Networks

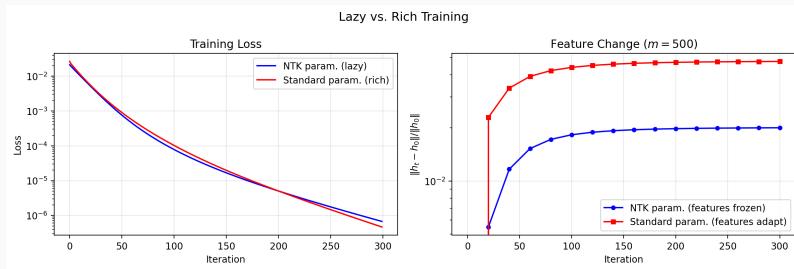
In finite-width networks, the NTK is *not* a perfectly constant matrix or kernel function. As training proceeds:

- The kernel **aligns** with the target function: Θ_t evolves to amplify directions that reduce loss fastest.
- This **kernel adaptation** can accelerate learning of task-relevant features beyond what the initial NTK predicts.
- The spectral bias still holds qualitatively, but the eigendirections are no longer purely frequency-ordered — they mix with **task-relevant structure**.

This is the bridge between the analytically tractable NTK regime (infinite width, frozen kernel) and the **feature learning** regime where networks develop task-specific representations.

Lazy vs. Rich Training

Numerical Demo: Lazy vs. Rich



Same network ($m = 500$), same learning rate. NTK parameterization: features barely move — **lazy**. Standard parameterization: features change by $O(1)$ — **rich**. The difference is purely in the output-layer scaling.

When Do Features Actually Change?

Consider a 2-layer net:

$$f(\mathbf{x}) = \frac{1}{m^\gamma} \sum_{i=1}^m a_i \sigma(\mathbf{w}_i^T \mathbf{x}), \quad \nabla_{\mathbf{w}_i} \mathcal{L} = \underbrace{\frac{a_i}{m^\gamma}}_{\text{factor } 1/m^\gamma} \sigma'(\mathbf{w}_i^T \mathbf{x}) (f(\mathbf{x}) - y) \mathbf{x}$$

If $\gamma \in [\frac{1}{2}, 1)$ with learning rate $\eta = \eta_0 \cdot m^{-2(1-\gamma)}$, the training is **lazy** for $m \rightarrow \infty$.

The standard network uses $\gamma = 0$ (actually $\gamma < \frac{1}{2}$); the above learning rate leads to the gradient flow as $m \rightarrow \infty$. It is not fully understood.

Theorem (Chizat et al., 2019)

As $m \rightarrow \infty$:

- $\frac{1}{2} \leq \gamma < 1$: **Lazy** regime. Features frozen, NTK is exact.
- $\gamma = 1$: **Rich** mean-field regime. Features evolve.

We can derive the corresponding limiting equation for these regimes.

Chizat, Oyallon, & Bach. *On Lazy Training in Differentiable Programming*. NeurIPS 2019.

Mean Field Theory

With $1/m$ scaling (not $1/\sqrt{m}$), hidden neurons behave like interacting particles:

$$f(x) = \frac{1}{m} \sum_{i=1}^m a_i \sigma(w_i^T x), \quad \mu_m = \frac{1}{m} \sum_{i=1}^m \delta_{(w_i, a_i)} \xrightarrow{m \rightarrow \infty} \mu(t)$$

The function becomes a **linear functional** of the neuron distribution:

$$f_\mu(x) = \int a \sigma(w^T x) d\mu(w, a)$$

Crucial shift: Optimization over $2m$ parameters (w_i, a_i) becomes optimization over an **infinite-dimensional probability measure** μ on $\mathbb{R}^{d+1}$.

Theorem (Mei, Montanari, Nguyen, 2018)

*The empirical measure $\mu_m(t)$ converges to the solution of a **Wasserstein gradient flow**:*

$$\partial_t \mu_t = \nabla_{(w,a)} \cdot (\mu_t \nabla_{(w,a)} \Psi[\mu_t](w, a))$$

where $\Psi[\mu](w, a) = \frac{\delta \mathcal{L}}{\delta \mu}(w, a)$ is the functional derivative of the loss.

Wasserstein Gradient Flow: Intuition

The PDE $\partial_t \mu_t = \nabla \cdot (\mu_t \nabla \Psi[\mu_t])$ is a **continuity equation** for probability mass.

- $\Psi[\mu](w, a)$ is a **potential** — neurons move downhill in this potential.
- The velocity field is $v(w, a) = -\nabla_{(w,a)} \Psi[\mu](w, a)$.
- Mass conservation: $\int \mu_t = 1$ for all t .

Each neuron (w_i, a_i) follows the **characteristic ODE** (like physical movement of particle):

$$\dot{w}_i = -\nabla_w \Psi[\mu_t](w_i, a_i), \quad \dot{a}_i = -\nabla_a \Psi[\mu_t](w_i, a_i)$$

This is exactly gradient descent on the original parameters when m is finite.

Convexity in Measure Space

A remarkable property: while $\mathcal{L}(\theta)$ is highly non-convex in the parameters θ , the functional $\mu \mapsto \mathcal{L}[f_\mu]$ is **convex** in μ . Overparameterization simplifies the landscape.

Consequences:

- **Global convergence:** the Wasserstein gradient flow always reaches a global minimum (under mild conditions).
- The non-convexity of parameter-space optimization is an artifact of the **parameterization** (permutation symmetry, scale invariance), not the optimization problem itself.
- The mean-field limit “lifts” non-convex optimization to convex optimization at the cost of infinite dimensionality.

Sources: Chizat & Bach (2018), Mei et al. (2018).

NTK vs. Mean-Field

	NTK	Mean-Field
Output scaling	$1/\sqrt{m}$	$1/m$
Parameter change	$O(1/\sqrt{m})$	$O(1)$
Feature change	$O(1/m) \rightarrow 0$	$O(1)$
Limit model	Kernel regression	Convex opt. over measures
Dynamics	Linear ODE	Nonlinear PDE (Wasserstein GF)
Feature learning?	No	Yes
Global convergence?	Yes (convex)	Yes (convex in μ)
Width needed	Larger	Smaller

The scaling is important NTK: $m \gg N$ (width $\gg$ dataset size). Mean-field: $m = O(N)$ (proportional regime).

Research Projects

Project 1: What Is a Deep Linear Network?

Definition: An L -layer network with **no activation functions**:

$$f(x) = W_L W_{L-1} \cdots W_1 x$$

Each weight matrix is $W_\ell \in \mathbb{R}^{2 \times 2}$.

The input-output map is just a matrix product — it's **linear**. So why study this?

The loss landscape is nonlinear: MSE gives a polynomial of degree $2L$ in the weights. For $L = 3$, this is a degree-6 polynomial in 12 variables.

Gradient flow dynamics are nonlinear: $\dot{W}_\ell = -\nabla_{W_\ell} \mathcal{L}$ couples all layers through the error signal.

This is the **simplest model** that exhibits nontrivial training dynamics — plateaus, acceleration, singular vector rotation — while remaining analytically tractable.

Project 1: The End-to-End Matrix

End-to-end matrix: $W = W_L W_{L-1} \cdots W_1 \in \mathbb{R}^{2 \times 2}$.

This matrix has only **4 degrees of freedom** — it's just a 2×2 matrix.

But the parameters live in $\mathbb{R}^{4L}$: each of the L matrices contributes 4 entries.

The “extra” $4(L - 1)$ dimensions encode **internal representations** — how the network factors the end-to-end map across layers. These internal degrees of freedom are invisible in the input-output behavior but determine the **training trajectory**.

The key tension: Many parameter configurations map to the same end-to-end matrix W . Gradient flow navigates this redundant parameterization. Understanding *how* it navigates is the research question.

Project 1: The Loss and Its Geometry

Loss: MSE on data (X, Y) with $X, Y \in \mathbb{R}^{2 \times n}$:

$$\mathcal{L}(W_1, \dots, W_L) = \frac{1}{2} \|Y - W_L \cdots W_1 X\|_F^2$$

Key property: The loss depends on the weights only through the product $W = W_L \cdots W_1$. So $\mathcal{L}$ is invariant under:

$$(W_\ell, W_{\ell+1}) \mapsto (W_\ell A, A^{-1} W_{\ell+1})$$

for any invertible $A \in \text{GL}(2)$. This is a **gauge symmetry**.

Consequence: The loss landscape has flat directions — continuous families of parameters with identical loss. These symmetries are key to understanding both the simplification (balancedness) and the complexity (singular vector dynamics) of the system.

Project 1: Gradient Flow Derivation

Gradient flow (continuous-time limit of GD, $\eta \rightarrow 0$):

$$\dot{W}_\ell = -\nabla_{W_\ell} \mathcal{L}$$

The gradient for layer ℓ involves a **forward-backward** structure:

$$\nabla_{W_\ell} \mathcal{L} = (W_L \cdots W_{\ell+1})^\top \underbrace{(WX - Y) X^\top}_{\text{error } E} (W_{\ell-1} \cdots W_1)^\top$$

Forward pass: $W_{\ell-1} \cdots W_1 X$ propagates the input to layer ℓ .

Backward pass: $(W_L \cdots W_{\ell+1})^\top E$ propagates the error back to layer ℓ .

The gradient is the outer product: **backward signal** $\otimes$ **forward activation**. This is exactly the backpropagation structure, made explicit.

Project 1: Balancedness — Definition

Definition (Balanced Initialization)

Weights satisfy the **balancedness condition** if for all $\ell = 1, \dots, L - 1$:

$$W_{\ell+1}^\top(0)W_{\ell+1}(0) = W_\ell(0)W_\ell^\top(0)$$

In words: the Gram matrix of each layer's rows equals the Gram matrix of the next layer's columns.

Why this condition?

- It holds when all layers are initialized with the **same variance** (e.g., Xavier init) and small weights.
- In the NTK regime ($m \rightarrow \infty$), it's automatically satisfied.
- For finite-width studies, it's the natural “neutral” starting point.

Project 1: Balancedness Is Preserved

Theorem (Preservation of Balancedness)

If $W_{\ell+1}^\top(0)W_{\ell+1}(0) = W_\ell(0)W_\ell^\top(0)$ for all ℓ , then this equality holds for all $t \geq 0$ under gradient flow.

Proof sketch: Differentiate the condition. The gradient flow dynamics give:

$$\frac{d}{dt}(W_{\ell+1}^\top W_{\ell+1} - W_\ell W_\ell^\top) = 0$$

when the condition holds. This is a **conservation law** of the dynamics.

Consequence: Balancedness is a stable submanifold of parameter space. Once you start there, you never leave. This dramatically reduces the effective dimensionality of the dynamics.

Saxe, McClelland, & Ganguli (2014), ICLR.

Project 1: Saxe's Theorem — Simplified Dynamics

Theorem (Saxe et al., 2014)

Under balancedness, the end-to-end matrix $W = W_L \cdots W_1$ evolves as:

$$\dot{W}(t) = -L \cdot (W(t)W(t)^\top)^{\frac{L-1}{L}} \nabla \mathcal{L}(W(t))$$

*The SVD of $W = U\Sigma V^\top$ diagonalizes the dynamics. Each singular value σ_i follows a **decoupled scalar ODE**. The singular vectors U, V of W remain **constant**.*

What this accomplishes: The full $4L$ -dimensional coupled dynamics collapse to just **two independent scalar ODEs** (for the two singular values of a 2×2 matrix). The remaining $4L - 2$ dimensions are frozen by balancedness.

Saxe, McClelland, & Ganguli (2014), ICLR.

Project 1: Implicit Bias — Arora's Theorem

Theorem (Arora et al., 2019)

Under gradient flow with balanced initialization, as $t \rightarrow \infty$:

$$W(t) \rightarrow W_\infty = \arg \min_W \|W\|_F \quad \text{s.t.} \quad WX = Y$$

*The deep linear network converges to the **minimum Frobenius-norm solution**.*

Why this is remarkable: The implicit bias is **independent of depth**. A 1-layer network (linear regression) and a 100-layer deep linear network converge to the *same* solution.

Depth doesn't change **where** you end up — only **how** you get there. The trajectory through parameter space depends strongly on L , but the destination does not.

Arora, Cohen, Hu, & Luo (2019), NeurIPS.

Project 1: Singular Value Dynamics — The ODE

Under balancedness, each singular value $\sigma_i(t)$ of $W(t)$ evolves via:

$$\dot{\sigma}_i = -L \cdot \sigma_i^{2-2/L} \cdot (\sigma_i - \sigma_i^*)$$

where σ_i^* are the singular values of W_∞ .

Key observations:

- The prefactor $\sigma_i^{2-2/L}$ means **smaller singular values converge slower**.
- This is a **spectral bias**: the network first learns the dominant singular modes, then refines the smaller ones.
- The exponent $2 - 2/L$ depends on depth: deeper networks have a stronger bias toward large singular values.

For $L = 2$: $\dot{\sigma}_i = -2\sigma_i(\sigma_i - \sigma_i^*)$ — exactly solvable in closed form.

Project 1: $L = 2$ — The Solved Case

For $L = 2$, the dynamics are completely understood:

$$\dot{\sigma}_i = -2\sigma_i(\sigma_i - \sigma_i^*), \quad i = 1, 2$$

Closed-form solution (for $\sigma_i^* > 0$):

$$\sigma_i(t) = \frac{\sigma_i^*}{1 + (\sigma_i^*/\sigma_i(0) - 1)e^{-2\sigma_i^*t}}$$

What this tells us:

- Each singular value follows **logistic growth** toward its target.
- The two singular modes are **completely independent**.
- The singular vectors of W never change — only the singular values evolve.
- The entire 8-dimensional dynamics reduce to 2 independent scalar ODEs.

For $L = 2$, the problem is **solved**. The frontier begins at $L = 3$.

Project 1: $L \geq 3$ — Why It's Harder

For $L \geq 3$, the singular values of W still decouple, but:

The internal singular vectors are not frozen. Each layer W_ℓ has its own SVD $W_\ell = U_\ell \Sigma_\ell V_\ell^\top$. The singular vectors U_ℓ, V_ℓ of *individual layers* can **rotate** during training, even though the singular vectors of the *end-to-end* matrix W stay fixed.

The coupling mechanism: U_ℓ (left singular vectors of W_ℓ) and $V_{\ell+1}$ (right singular vectors of $W_{\ell+1}$) interact. They tend to **align** — U_ℓ rotates toward $V_{\ell+1}^\top$. This alignment affects the effective step size for each singular mode.

For $L = 4$, the 16-dimensional system has enough freedom to potentially exhibit **oscillatory** or even **chaotic** transient behavior.

Project 1: Open Problem 1.1 — The Geometry

Setup: $W_1, W_2, W_3 \in \mathbb{R}^{2 \times 2}$, generic (non-balanced) initial conditions, gradient flow.

SVD of each layer: $W_\ell = U_\ell \Sigma_\ell V_\ell^\top$.

Each $U_\ell, V_\ell \in O(2)$ is parameterized by a single angle (plus a chirality bit for rotation vs. reflection):

$$U_\ell = \begin{pmatrix} \cos \theta_\ell & -\sin \theta_\ell \\ \sin \theta_\ell & \cos \theta_\ell \end{pmatrix}$$

The dynamics reduce to **coupled ODEs for 6 angles:** $\theta_{U_1}, \theta_{V_1}, \theta_{U_2}, \theta_{V_2}, \theta_{U_3}, \theta_{V_3}$.

Modulo gauge symmetry $W_\ell \rightarrow A_\ell W_\ell A_{\ell-1}^{-1}$, only **4 relative angles** are physical.

Saxe et al. (2014); Arora et al. (2019).

Project 1: Open Problem 1.1 — The Questions

Goal: Fully characterize the singular vector rotation dynamics.

1. **Derive the coupled ODE system** for the 4 physical relative angles. What is the explicit form of $\dot{\theta}_{U_1} - \dot{\theta}_{V_2}$, etc.?
2. **Identify all fixed points.** Which ones correspond to aligned singular vectors ($U_\ell \parallel V_{\ell+1}^\top$)? What is the stability of each fixed point?
3. **Prove alignment or find counterexamples.** Do the singular vectors always align asymptotically? Or are there stable misaligned equilibria where the vectors lock at a nonzero relative angle?

Project 1: Open Problem 1.1 — Approach & Impact

Approach: Derive the full ODE system in angle coordinates. Use numerical integration to map the phase portrait. Then prove stability analytically via linearization and Lyapunov functions.

Why this matters:

- Misalignment between layers creates a **bottleneck** — singular values stall until vectors align.
- Understanding alignment dynamics could explain **plateaus** in deep network training.
- The $2 \times 2 \times 2$ case is the minimal model for studying representation alignment in deep learning.

Saxe et al. (2014); Arora et al. (2019).

Project 1: Open Problem 1.2 — Limit Cycles

Setup: $W_1, W_2, W_3, W_4 \in \mathbb{R}^{2 \times 2}$. Parameter space $\mathbb{R}^{16}$. Modulo gauge: ~ 8 – 10 effective dimensions.

The phenomenon: Numerical experiments suggest that for badly-conditioned targets, singular values can **oscillate** — they overshoot, reverse direction, and converge non-monotonically.

The question: Prove or disprove the existence of limit cycles.

Why this would be surprising: Gradient flow on an analytic loss function typically can't have limit cycles — the loss is a global Lyapunov function (it always decreases). But the internal degrees of freedom (singular vectors) could potentially cycle while the loss still decays.

Project 1: Open Problem 1.2 — Strategy

How to attack this:

1. **Numerical search:** Use continuation software (AUTO, MatCont) to track periodic solutions. Look for **Hopf bifurcations** — complex-conjugate eigenvalue pairs crossing the imaginary axis.
2. **Center manifold reduction:** Near a Hopf point, reduce the 10-dim dynamics to a 2-dim center manifold. The normal form determines supercritical vs. subcritical bifurcation.
3. **Lyapunov exponents:** Even without true limit cycles, compute the maximal Lyapunov exponent to detect **chaotic transients**.
4. **Topological constraints:** Apply Poincaré-Bendixson theory (or higher-dimensional generalizations) to constrain possible ω -limit sets given the global Lyapunov function.

Project 1: Open Problem 1.3 — Complex Entries

Setup: $W_\ell \in \mathbb{C}^{2 \times 2}$ for $\ell = 1, \dots, L, L \geq 3$. MSE loss with complex matrices.

Why complex?

- Complex numbers have **phase** degrees of freedom: $z = re^{i\phi}$ has both magnitude r and phase ϕ .
- Many applications use complex networks: signal processing (Fourier domain), quantum ML, wireless communications.
- The complex case is often mathematically **cleaner** — every real matrix has complex eigenvalues.

Balancedness: $W_{\ell+1}^* W_{\ell+1} = W_\ell W_\ell^*$, where $*$ is conjugate transpose.

The new phenomenon: Complex phases can rotate **independently** of singular values. The balancedness condition leaves a $U(1)^{2L}$ phase gauge symmetry.

Project 1: Open Problem 1.3 — Questions

Specific goals:

1. Convert to a real dynamical system on $\mathbb{R}^{8L}$ via realification (separating real and imaginary parts). The phase dynamics become an $O(2)^L$ coupled oscillator system.
2. **Characterize the phase dynamics.** Are there **invariant tori** where phases circulate indefinitely while singular values converge?
3. Look for **conserved quantities** (Hamiltonians) in the phase subspace. The balancedness condition may imply additional conservation laws beyond the real case.
4. Determine whether phase rotation **delays** convergence (the phases must settle before singular values converge) or is **independent** (singular values converge regardless of phase state).

Project 1: Open Problem 1.4 — Depth Scaling

Setup: Generic $W_\ell \in \mathbb{R}^{2 \times 2}$, depth $L \gg 1$. Convergence time $T_{\text{conv}} \sim O(L^p)$.

Empirical observation: $p \approx 1.5\text{--}2.5$, depending on initialization and target conditioning.

The question: Determine the precise exponent p and its dependence on:

- Initialization variance σ_w^2 — does p jump at $\sigma_w^2 = 1$ (ordered/critical/chaotic transition)?
- Target condition number $\kappa^* = \sigma_{\max}^* / \sigma_{\min}^*$.
- Whether p is smooth in L or has discrete jumps at specific depths.

Project 1: Open Problem 1.4 — Approach

Analytical strategy: Linearize the dynamics around W_∞ . The **slowest eigenvalue** of the linearized operator determines p .

For the Arora ODE $\dot{\sigma}_i = -L\sigma_i^{2-2/L}(\sigma_i - \sigma_i^*)$, linearize around $\sigma_i = \sigma_i^*$:

$$\dot{\delta}_i = -L(\sigma_i^*)^{2-2/L} \cdot \delta_i$$

The convergence rate is $\sim L(\sigma_{\min}^*)^{2-2/L}$. As $L \rightarrow \infty$, if $\sigma_{\min}^* < 1$, the exponent $2 - 2/L \rightarrow 2$, and convergence becomes $\sim L(\sigma_{\min}^*)^2$ — linear in L .

Numerical strategy: Train networks for $L = 2, 3, \dots, 100$ with controlled initialization and targets. Fit $T_{\text{conv}} = c \cdot L^p$ to extract p .

Saxe et al. (2014); Arora et al. (2019); Schoenholz et al. (2017).

Project 2: The Optimization Problem

Goal: Minimize $f : \mathbb{R}^d \rightarrow \mathbb{R}$ using only gradient queries.

Assumptions:

- f is L -smooth: $\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$ for all x, y .
- f is μ -strongly convex: $f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2}\|y - x\|^2$.

Condition number: $\kappa = L/\mu \geq 1$. Measures how “elliptical” the level sets are. $\kappa = 1$ means spherical (easy). $\kappa \gg 1$ means highly elongated (hard).

Gradient descent with predetermined step sizes $\eta_0, \eta_1, \dots$:

$$w_{k+1} = w_k - \eta_k \nabla f(w_k)$$

No momentum. No line search. No gradient history. Just pure GD with a clever schedule.

Project 2: Fixed-Step GD — The Baseline

Fixed step size $\eta = 1/L$ (the largest step guaranteed to not diverge):

$$\|w_k - w^*\| \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^k \|w_0 - w^*\|$$

Iteration complexity: To reach accuracy ε :

$$K \geq \kappa \cdot \log(1/\varepsilon)$$

Linear convergence, but the constant is κ . When $\kappa = 1000$, you need ~ 7000 iterations for $\varepsilon = 10^{-3}$.

Why $\eta = 1/L$? It's the largest step that guarantees $\|I - \eta \nabla^2 f\| \leq 1$ for all points. Any larger, and GD can diverge on some functions. But **not every step needs to be this conservative**.

Project 2: Nesterov's Accelerated Gradient

Nesterov (1983) introduced **momentum**:

$$w_{k+1} = y_k - \eta \nabla f(y_k), \quad y_{k+1} = w_{k+1} + \beta_k (w_{k+1} - w_k)$$

where $\beta_k = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$.

Convergence rate:

$$\|w_k - w^*\| \leq O\left(\left(1 - \frac{1}{\sqrt{\kappa}}\right)^k\right)$$

Iteration complexity: $K \geq \sqrt{\kappa} \cdot \log(1/\varepsilon)$. For $\kappa = 1000$: ~ 100 iterations vs. 7000 for fixed-step GD.

This is **provably optimal** — no first-order method can beat $O(\sqrt{\kappa})$ on all smooth strongly convex functions.

Project 2: The Gap — And the Question

Method	Iterations $\sim$	Momentum?	Optimal?
Fixed-step GD	$\kappa^{1.0}$	No	No
Variable-step GD	?	No	?
Nesterov AGD	$\kappa^{0.5}$	Yes	Yes

The question: Where does variable-step GD (no momentum) sit on this spectrum?

Can we beat $O(\kappa)$ without momentum? How close can we get to the Nesterov rate $O(\sqrt{\kappa})$? And what is the **optimal** schedule?

Answer (Altschuler & Parrilo, 2023): Yes — and the answer involves the silver ratio $1 + \sqrt{2}$.

Project 2: Quadratics — The Polynomial Connection

Restrict to quadratics: $f(w) = \frac{1}{2}w^\top Aw - b^\top w$, $\lambda(A) \subseteq [\mu, L]$.

The error $e_k = w_k - w^*$ evolves linearly:

$$e_{k+1} = (I - \eta_k A) e_k$$

After K steps:

$$e_K = \underbrace{\left[\prod_{k=0}^{K-1} (I - \eta_k A) \right]}_{p_K(A)} e_0$$

$p_K(\lambda) = \prod_{k=0}^{K-1} (1 - \eta_k \lambda)$ is a degree- K polynomial with $p_K(0) = 1$.

The key reduction: Minimizing $\|e_K\|$ over all step size choices = minimizing $\max_{\lambda \in [\mu, L]} |p_K(\lambda)|$ over all degree- K polynomials with $p_K(0) = 1$.

This is **polynomial approximation theory**.

Project 2: Chebyshev Polynomials

Chebyshev polynomials of the first kind:

$$T_K(x) = \cos(K \arccos x), \quad x \in [-1, 1]$$

The minimax property: Among all degree- K polynomials with leading coefficient 2^{K-1} , T_K has the **smallest maximum absolute value** on $[-1, 1]$:

$$\max_{x \in [-1, 1]} |T_K(x)| = 1$$

Why this is optimal: Any other polynomial with the same leading coefficient must exceed ± 1 somewhere — T_K oscillates between exactly $+1$ and -1 at $K + 1$ points (the **equioscillation property**).

Project 2: Chebyshev Step Sizes for Quadratics

Map $[\mu, L] \rightarrow [-1, 1]$ via $\lambda \mapsto \frac{L+\mu-2\lambda}{L-\mu}$. The optimal residual polynomial is:

$$p_K^*(\lambda) = \frac{T_K\left(\frac{L+\mu-2\lambda}{L-\mu}\right)}{T_K\left(\frac{L+\mu}{L-\mu}\right)}$$

Theorem (Chebyshev-Optimal Step Sizes)

The roots of p_K^ give the step sizes:*

$$\eta_k = \frac{2}{L + \mu - (L - \mu) \cos\left(\frac{2k+1}{2K} \pi\right)}, \quad k = 0, \dots, K-1$$

These achieve:

$$\|w_K - w^*\| \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^K \|w_0 - w^*\|$$

This matches Nesterov's rate — **with no momentum**. But only for quadratics.

Project 2: Why Chebyshev Fails for General Functions

The problem: Chebyshev steps are optimal for a **fixed** quadratic. For a general convex function, the Hessian $\nabla^2 f(w)$ changes along the trajectory.

At w_0 , the local quadratic approximation has condition number κ_0 . The Chebyshev steps work for this κ_0 . But at w_k , the local curvature may differ — the step sizes that were optimal near w_0 may be **too aggressive** (causing divergence) or **too conservative** (wasting iterations) near w_k .

The challenge: Design a step size schedule that is **robust** to variation in the local condition number. The schedule must work for *any* function in the class, not just one quadratic.

This requires a fundamentally different construction than polynomial roots.

Project 2: The Silver Schedule — Theorem

Theorem (Altschuler & Parrilo, 2023)

For any L -smooth μ -strongly convex function, GD with a recursively-defined, non-monotonic schedule achieves:

$$K = \kappa^{\log_{\rho} 2} \cdot \text{poly}(\log(1/\varepsilon)), \quad \rho = 1 + \sqrt{2}$$

The exponent: $\log_{\rho} 2 = \frac{\log 2}{\log(1+\sqrt{2})} \approx 0.7864$.

The silver ratio $\rho = 1 + \sqrt{2} \approx 2.414$ satisfies $\rho - 1/\rho = 2$, analogous to the golden ratio $\phi - 1/\phi = 1$. Its continued fraction is $[2; 2, 2, \dots]$.

The schedule is non-monotonic (step sizes go up and down) and **fractal-like** (self-similar across scales). It is built by recursive doubling from the base case.

Project 2: Where the Silver Schedule Sits

$$\underbrace{\kappa^{0.5}}_{\text{Nesterov (momentum)}} < \underbrace{\kappa^{0.7864}}_{\text{Silver (no momentum)}} < \underbrace{\kappa^{1.0}}_{\text{Fixed-step GD}}$$

The gap: $0.7864 - 0.5 = 0.2864$. This is the “cost of no momentum” — the best known rate without momentum is $\kappa^{0.2864}$ times worse than with momentum.

For non-strongly convex functions ($\mu = 0$, only L -smooth):

$$\underbrace{\varepsilon^{-0.5}}_{\text{Nesterov}} < \underbrace{\varepsilon^{-0.7864}}_{\text{Silver}} < \underbrace{\varepsilon^{-1.0}}_{\text{Fixed-step GD}}$$

Is this exponent optimal? That’s Open Problem 2.1.

Project 2: Recursive Construction — The Doubling Step

Define normalized stepsizes y_n, z_n . Start: $y_1 = z_1 = 1/\kappa$.

Doubling ($n \rightarrow 2n$):

$$y_{2n}z_{2n} = z_n^2, \quad z_{2n} - y_{2n} = 2(z_n - z_n^2)$$

Two equations uniquely determine y_{2n}, z_{2n} from z_n . The first equation enforces a **product constraint**. The second enforces a **difference constraint**.

Explicit solution: $z_{2n} = z_n(\xi + \sqrt{1 + \xi^2})$ where $\xi = 1 - z_n$.

For small z_n (large κ , early iterations): $\xi \approx 1$, growth factor $\approx 1 + \sqrt{2} = \rho$. This is where the silver ratio emerges.

Project 2: From y_n, z_n to Actual Step Sizes

The normalized stepsizes y_n, z_n are **not** the actual step sizes — they are building blocks.

Linear fractional transformation:

$$\eta_k = \frac{1}{L} + \frac{L - \mu}{L\mu} \cdot \frac{(\text{function of } y_n, z_n \text{ for appropriate } n, k)}{1 + (\text{correction term})}$$

The transformation maps the normalized interval to $[1/L, 1/\mu]$. The actual schedule is:

- **Non-monotonic:** Short steps ($\approx 1/L$) alternate with long steps ($\approx 1/\mu$).
- **Fractal:** The pattern at scale n contains compressed copies of the pattern at smaller scales.
- **Deterministic:** No adaptivity — the entire schedule is fixed before optimization begins.

Project 2: Two-Phase Convergence

The Silver schedule exhibits a sharp **phase transition** at $n^* = \Theta(\kappa^{\log_\rho 2})$:

Phase 1 — Acceleration ($n \leq n^*$):

$$\|w_n - w^*\|^2 \leq \exp\left(-\Theta\left(\frac{n \log^2 \rho}{\kappa}\right)\right)$$

Super-exponential convergence — faster than any exponential. This phase uses very small steps, exploiting local quadratic structure.

Phase 2 — Saturation ($n > n^*$):

$$\|w_n - w^*\|^2 \leq \exp\left(-\Theta\left(\frac{n}{\kappa^{\log_\rho 2}}\right)\right)$$

Exponential convergence with the improved condition number. The acceleration saturates.

Project 2: The Hedging Intuition

Short steps ($\eta \approx 1/L$) and **long steps** ($\eta \approx 1/\mu$) have complementary failure modes.

- A function that's “bad” for short steps (requires many iterations because each step makes minimal progress) is “good” for long steps (each long step covers substantial distance in the well-conditioned directions).
- A function that's “bad” for long steps (risks divergence because the curvature varies rapidly) is “good” for short steps (they track the local geometry safely).

Hedging: By **interleaving** short and long steps, no single function is simultaneously the worst case for all steps. The recursive structure enforces **consistency** — the algebraic constraints $(y_{2n}z_{2n} = z_n^2)$ guarantee that iterates remain controlled across the entire trajectory.

Project 2: Grimmer's Long Steps — SDP Approach

Grimmer, Shu, & Wang (2024) use a fundamentally different method: **computer-assisted proofs via SDP**.

The PEP framework (Performance Estimation Problem):

- For GD with fixed step sizes $\eta_1, \dots, \eta_K$ and function class (L -smooth convex), the exact worst-case performance is computable by solving a **Semidefinite Program**.
- The SDP variables encode hypothetical function values and gradients at each iterate.
- A feasible dual solution **is** a rigorous proof of the convergence rate.

Result: Periodic patterns of long steps achieve the same exponent $\log_\rho 2$ as Silver, but with **smaller constant factors**. The step sizes periodically exceed $2/L$ (the stability threshold), temporarily increasing the objective.

Project 2: Composition Theory

Grimmer, Shu, & Wang (arXiv:2410.16249) unify both approaches:

1. **Basic composable schedule:** Two copies can be concatenated while preserving the multi-step descent guarantee.
2. The **Silver schedule is basic s -composable** — the $y_{2n}z_{2n} = z_n^2$ relation is exactly the algebraic condition for composability.
3. **Optimized schedules of every length** generalize the exponentially-spaced construction.

Zhang & Jiang (2024): Independent concatenation technique with analytic bounds for arbitrary iteration counts.

Convergence of methods: Three distinct approaches (Silver recursive, SDP long steps, concatenation) all yield the **same exponent** $\log_\rho 2$. This suggests either optimality or a shared limitation.

Project 2: Open Problem 2.1 — The Statement

Conjecture (Altschuler & Parrilo): The exponent $\log_\rho 2 \approx 0.7864$ is **optimal** for momentum-free GD with any deterministic step size schedule.

What's at stake:

- If **true**: Establishes a fundamental separation — momentum is **provably necessary** to achieve the Nesterov rate. No amount of step-size engineering can close the gap.
- If **false**: There exists a better schedule with exponent < 0.7864 , possibly significantly closer to 0.5. Finding it would be a landmark result in optimization.

Project 2: Open Problem 2.1 — Proof Strategies

Three approaches:

1. **Lower bounds via PEP duality:** The SDP formulation has a dual. A feasible dual solution gives a **lower bound** valid for *any* schedule. Can the dual be solved analytically?
2. **Information-theoretic arguments:** Adapt classical lower bounds (Nemirovski & Yudin) to the restricted class of deterministic, momentum-free step sequences. The restriction creates a smaller algorithm class — the $\Omega(\sqrt{\kappa})$ bound for general first-order methods may not apply.
3. **Construct a counterexample:** Use numerical optimization to search over non-recursive schedules. If a schedule with exponent < 0.7864 is found on a known hard function, the conjecture is false.

Altschuler & Parrilo (2023, 2024).

Project 2: Open Problem 2.2 — Stochastic Extension

The limitation: Silver/long-step theory covers only **deterministic, convex, smooth** optimization. Neural network training is none of these.

SGD setting: $w_{k+1} = w_k - \eta_k g_k$, where $\mathbb{E}[g_k] = \nabla f(w_k)$, $\text{Var}(g_k) = \sigma^2/B$.

Key challenge: Gradient noise creates a **diffusion term** — the error no longer contracts deterministically. The multi-step descent certificate framework needs to be extended to handle **probabilistic** guarantees.

Potential approach: Model the noise as a perturbation to the deterministic dynamics. Use **stochastic Lyapunov analysis** — show that the expected value of the Lyapunov function decreases despite noise.

Project 3: The Central Question

When $P \gg N$ (more parameters than data), infinitely many weight configurations achieve zero training error. **Gradient descent picks one.** Which one?

Why this matters:

- **Generalization:** The chosen solution determines test error. If we understand the bias, we can predict when networks will generalize.
- **Architecture design:** If we know the bias, we can design architectures whose bias matches the data structure.
- **Theory $\rightarrow$ practice:** Understanding implicit bias could eliminate the need for explicit regularization.

Current status: The bias is well-understood in two extreme limits (infinite width, lazy or rich). The regime that matters — finite width with feature learning — is open.

Project 3: A Spectrum of Known Results

Model	Bias	Reference
Linear regression, $w_0 = 0$	Min $\ w\ _2$	Standard
Matrix factorization	Min $\ W\ _*$ (nuclear)	Gunasekar et al. (2017)
Deep linear nets (balanced)	Min $\ W\ _F$	Arora et al. (2019)
NTK regime ($m \rightarrow \infty$, lazy)	Min $\ f\ _{\mathcal{H}_\Theta}$ (RKHS)	Jacot et al. (2018)
Mean-field 2L ($m \rightarrow \infty$, rich)	Max margin in μ -space	Chizat & Bach (2020)
Finite-width 2L ReLU	???	Open

The first five rows are limiting cases. Real networks operate in the last row.

Project 3: NTK Regime — What We Know

Assumptions: $m \rightarrow \infty$, output scaled by $1/\sqrt{m}$, $\eta = O(1)$.

Key facts:

- The NTK is **constant**: $\Theta_t \rightarrow \Theta_\infty$.
- Features never change: $\phi_i(x) = \frac{1}{\sqrt{m}}\sigma(w_i^\top x)$ are frozen.
- Training = kernel ridge regression with the NTK kernel.

Implicit bias: GD converges to the minimum RKHS norm interpolant:

$$f_\infty = \arg \min_{f \in \mathcal{H}_{\Theta_\infty}} \|f\|_{\mathcal{H}_{\Theta_\infty}} \quad \text{s.t.} \quad f(x_i) = y_i \quad \forall i$$

This is well-understood, computable, and explains spectral bias. The limitation: **no feature learning**.

Project 3: NTK Regime — The Gap

What the NTK theory doesn't capture:

- Real networks learn **data-adapted representations**. Lower layers develop edge detectors, texture analyzers, shape recognizers that are specific to the training distribution.
- The NTK is determined by the architecture at initialization and **never adapts**. It can't explain why transfer learning works (features learned on ImageNet help on CIFAR-10).
- Finite-width networks show **kernel adaptation** — the NTK evolves to amplify task-relevant directions. This is invisible in the infinite-width limit.

The NTK is a theory of what happens when nothing interesting happens. It explains the first few iterations of training remarkably well. It does not explain the feature learning that makes deep learning powerful.

Project 3: Mean-Field Regime — Setup

Assumptions: $m \rightarrow \infty$, output scaled by $1/m$, $\eta \propto m$.

Model: $f(x) = \frac{1}{m} \sum_{j=1}^m a_j \sigma(w_j^\top x)$, 2-layer ReLU.

The neuron distribution: As $m \rightarrow \infty$:

$$\mu_m(t) = \frac{1}{m} \sum_{j=1}^m \delta_{(w_j(t), a_j(t))} \xrightarrow{m \rightarrow \infty} \mu(t)$$

Instead of $2m(d+1)$ individual parameters, we track a single **probability measure** on $\mathbb{R}^{d+1}$.

Key shift: The dynamics of μ are governed by a **PDE** (Wasserstein gradient flow), not a finite-dimensional ODE.

Mei et al. (2018); Chizat & Bach (2018, 2020).

Project 3: The Chizat–Bach Theorem

Theorem (Chizat & Bach, 2020)

For 2-layer ReLU on separable binary classification, in the mean-field limit:

1. μ_t evolves via **Wasserstein gradient flow** of $\mathcal{L}(\mu)$.
2. As $t \rightarrow \infty$ (with rescaling), $\mu_t \rightarrow \mu_\infty$, where:

$$\mu_\infty \in \arg \min_{\rho} \left\| \int a \sigma(w^\top x) d\rho(a, w) \right\|_{TV} \text{ s.t. } y_i f_\rho(x_i) \geq 1$$

The rich-regime answer: The network selects the neuron distribution that **maximizes the classification margin**, where margin is measured by the total variation norm of the signed measure $a d\rho(a, w)$.

Project 3: Why The Proof Works — Linearity in μ

Step 1: Write the network output as a linear functional:

$$f_\mu(x) = \int a \sigma(w^\top x) d\mu(a, w)$$

The map $\mu \mapsto f_\mu$ is **linear**. This is the crucial structural insight.

Step 2: For any finite m , the network is exactly this model at the empirical measure:

$$f(x) = \frac{1}{m} \sum_{j=1}^m a_j \sigma(w_j^\top x) = \int a \sigma(w^\top x) d\mu_m(a, w)$$

where $\mu_m = \frac{1}{m} \sum \delta_{(w_j, a_j)}$.

Step 3: The loss $\mathcal{L}(\mu) = \frac{1}{N} \sum \ell(y_i f_\mu(x_i))$ is **convex in μ** — it's a convex function of a linear functional.

The non-convexity of training is an artifact of the **particle parameterization** $\mu = \frac{1}{m} \sum \delta_{(w_j, a_j)}$.

Project 3: Why The Proof Works — Wasserstein GF

Step 4: The finite- m gradient flow:

$$\dot{w}_j = -\nabla_{w_j} \mathcal{L}, \quad \dot{a}_j = -\nabla_{a_j} \mathcal{L}$$

is a **particle system** — each neuron follows the gradient of the loss.

Step 5: As $m \rightarrow \infty$, this particle system converges to the **Wasserstein gradient flow** PDE:

$$\partial_t \mu_t = \nabla_{(w,a)} \cdot (\mu_t \nabla_{(w,a)} \Psi[\mu_t](w, a))$$

where $\Psi[\mu](w, a) = \frac{\delta \mathcal{L}}{\delta \mu}$ is the functional derivative.

Step 6: For convex $\mathcal{L}(\mu)$, the Wasserstein gradient flow has a unique global minimizer, characterized by the KKT conditions of the convex problem.

Project 3: The Finite-Width Gap — Fluctuations

At finite m , three effects break the infinite-width theory:

1. Finite-particle fluctuations: $\mu_m(t) = \mu(t) + O(1/\sqrt{m})$.

- The empirical measure of m particles is a **noisy** approximation to the PDE solution.
- Each neuron feels the mean field of all other neurons plus an $O(1/\sqrt{m})$ individual fluctuation.
- Over $O(m)$ time steps, fluctuations accumulate and can **change which minimum** is selected.
- Analogous to **finite-size effects** in statistical physics — the thermodynamic limit ($m \rightarrow \infty$) can hide phenomena visible only at finite system size.

Project 3: The Finite-Width Gap — Discretization & LR

2. Discretization of the feature space:

- Infinite- m : Can represent **any** distribution over ReLU features.
- Finite- m : Restricted to measures supported on $\leq m$ atoms.
- The infinite-dimensional optimizer μ_∞ may require continuous support — finite m introduces **approximation error** scaling as $O(1/m)$.

3. Width-LR interaction:

- At infinite m : Lazy ($\eta = O(1)$) vs. rich ($\eta = O(m)$) is a sharp transition.
- At finite m : It's a **crossover**. Intermediate η gives **partial** feature learning.
- The implicit bias may interpolate between RKHS norm and TV norm, with interpolation weight $\lambda = \lambda(m, \eta)$.

Project 3: Open Problem 3.1 — Directional Convergence

Part 1: For a finite-width 2-layer ReLU net trained with gradient flow on separable data:

$$\text{Does } \lim_{t \rightarrow \infty} \frac{\theta(t)}{\|\theta(t)\|} \text{ exist?}$$

What's known:

- Linear models: Yes — the limit is the minimum ℓ_2 -norm solution.
- Deep linear nets (balanced): Yes (Arora et al. 2019).
- 2-layer ReLU, mean-field limit ($m \rightarrow \infty$): Yes (Chizat & Bach 2020).
- 2-layer ReLU, **finite m : Unknown.**

The obstacle: At finite m , the dynamics are a nonlinear ODE in $2m(d + 1)$ dimensions. There is no known Lyapunov function enforcing directional convergence. Parameters could drift along the zero-loss manifold indefinitely.

Project 3: Open Problem 3.1 — The Bias Functional

Part 2: If directional convergence holds, characterize the limit.

Expected structure:

$$\theta_\infty \in \arg \min \|\theta\|_P \quad \text{s.t.} \quad y_i f(x_i; \theta) \geq 1$$

for some **norm** (or gauge function) $\|\cdot\|_P$.

Known limiting cases:

- $m \rightarrow \infty, \eta = O(1)$ (lazy): $\|\theta\|_P \rightarrow \|\cdot\|_{\mathcal{H}_{\Theta_\infty}}$ (RKHS norm).
- $m \rightarrow \infty, \eta = O(m)$ (rich): $\|\theta\|_P \rightarrow \|\cdot\|_{\text{TV}}$ (TV norm on measures).

The goal: A formula for $\|\theta\|_P$ as a function of m and η , valid across the full lazy-to-rich spectrum.

Project 3: Open Problem 3.1 — The Phase Transition

Part 3: Characterize the lazy-rich crossover at finite m .

Key questions:

1. Is there a **sharp phase transition** at some critical $\eta_c(m)$, or is the crossover smooth?
2. If sharp: Does $\eta_c(m) \sim m^{\alpha_c}$ for some universal α_c ?
3. On the lazy side: Does the bias functional expand as $\|\theta\|_P = \|\theta\|_{\mathcal{H}} + \frac{1}{m}\|\theta\|_{\text{corr}} + O(1/m^2)$?
4. Does the transition correspond to a qualitative change in neuron behavior — from collective motion (lazy) to specialization (rich)?

Approach: Numerical experiments tracking neuron trajectories at varying (m, η) . Look for signatures of critical phenomena: diverging correlation length, power-law scaling, universal exponents.

Project 3: Open Problem 3.2 — The Role of Depth

The challenge: For $L \geq 3$, features are **hierarchical**. Layer ℓ 's features depend on layer $\ell - 1$'s features, which depend on layer $\ell - 2$'s, etc.

$$f(x) = W_L \sigma(W_{L-1} \sigma(\cdots \sigma(W_1 x)))$$

Why this breaks the 2-layer analysis: For a 2-layer net, there is one hidden layer $\rightarrow$ one neuron distribution $\mu^{(1)}$. For an L -layer net, the state is a **hierarchy** of conditional distributions $\mu^{(1)}, \mu^{(2|1)}, \dots, \mu^{(L|L-1)}$. The number of coupled degrees of freedom grows combinatorially.

The SVD trick from Saxe (deep linear) does not carry over: ReLU breaks linearity — you can't diagonalize the dynamics with a single change of basis.

Project 3: Open Problem 3.2 — Candidate Tools

Three possible approaches:

1. **Tensor Programs (Yang, 2019–2023):** Provides a rigorous calculus for infinite-width limits of arbitrary architectures. Currently handles the NTK (lazy) regime via the Master Theorem. **Can it be extended to the rich regime** by adjusting the scaling laws?
2. **Hierarchical mean-field PDEs:** Model each layer's neuron distribution $\mu^{(\ell)}$ as evolving under the influence of the layer below. Creates a **forward-backward PDE system** — forward for activations, backward for gradients.
3. **Field-theoretic methods:** Borrow from statistical physics — dynamical mean-field theory, cavity method, replica theory. These tools handle precisely the kind of disordered, high-dimensional, nonlinear dynamics that deep networks exhibit.

Project 4: The Setup

Model: 1D shallow ReLU network, slopes fixed at $w_j = 1$:

$$f(x) = \sum_{j=1}^m a_j \sigma(x - b_j), \quad \sigma(z) = \max(0, z), \quad x \in [-1, 1]$$

This is a **piecewise linear** function. Between bias points, the function is linear with a fixed slope. At each b_j , the slope changes by a_j .

Training: Gradient flow on MSE loss over the continuous domain:

$$\mathcal{L} = \frac{1}{2} \int_{-1}^1 (f(x) - f^*(x))^2 dx$$

where $f^*(x)$ is a smooth target (e.g., sum of sinusoids).

The surprise: Even with $m = 100$ neurons, the biases b_j **collapse** to only ~ 6 – 8 distinct values. The network discovers it only needs a few kinks.

Project 4: The Gradient Flow ODEs — Amplitudes

Amplitude dynamics:

$$\dot{a}_j = -\frac{\partial \mathcal{L}}{\partial a_j} = -\int_{-1}^1 (f(x) - f^*(x)) \sigma(x - b_j) dx$$

Interpretation: $\dot{a}_j$ is the **negative correlation** between the residual error $f - f^*$ and the ReLU basis function $\sigma(x - b_j)$.

- If the residual is positive where $\sigma(x - b_j)$ is large (to the right of b_j), then a_j decreases.
- If the residual is negative there, a_j increases.
- The amplitude adjusts to reduce the projected error along that ReLU feature.

Project 4: The Gradient Flow ODEs — Biases

Bias dynamics:

$$\dot{b}_j = -\frac{\partial \mathcal{L}}{\partial b_j} = a_j \int_{-1}^1 (f(x) - f^*(x)) \sigma'(x - b_j) dx = a_j \int_{b_j}^1 (f(x) - f^*(x)) dx$$

Interpretation: $\dot{b}_j$ is proportional to the **integrated residual** to the right of b_j , weighted by the neuron's amplitude a_j .

Key insight: If two biases b_i and b_j are close, they see nearly the same integrated residual:

$$\int_{b_i}^1 (f - f^*) dx \approx \int_{b_j}^1 (f - f^*) dx$$

Their dynamics **synchronize** — they move together.

Project 4: The Empirical Phenomenon

Observation (supported by numerical experiments, formal proof is Open Problem 4.1):

Let $f^*(x)$ be a target on $[-1, 1]$. Let k be the number of kinks in the optimal m -term piecewise linear L^2 -approximation of f^* . If $m > k$, then under gradient flow:

1. The biases $b_j(t)$ appear to converge to at most k distinct limit points.
2. The amplitudes of biases sharing a limit point sum to the combined weight.
3. The network output converges to the optimal k -kink approximation of f^* .

Status: This is a **conjecture** supported by numerical evidence. Proving it — the cluster count, convergence, and error bounds — is the goal of Open Problems 4.1–4.3.

Chapter 11. Formal proof is open — this is the research contribution expected from the project.

Project 4: Why Collapse Happens

Fundamental reason: A ReLU network with sorted biases $b_1 < \dots < b_m$ is a piecewise linear function with kinks at the b_j . If the target f^* can be well-approximated with k kinks, then $m - k$ neurons are redundant.

The redundancy is explicit: Two biases at the same location $b_i = b_j$ with amplitudes a_i, a_j produce **exactly** the same function as one bias at b_i with amplitude $a_i + a_j$:

$$a_i \sigma(x - b_i) + a_j \sigma(x - b_i) = (a_i + a_j) \sigma(x - b_i)$$

Gradient flow exploits this: The loss landscape has **flat directions** corresponding to moving redundant biases. Along these directions, the loss doesn't change (to first order). Second-order effects drive biases toward the active ones.

Project 4: Synchronization — Mathematical Analysis

Suppose b_i and b_j are close: $|b_i - b_j| = \varepsilon \ll 1$.

Difference in bias velocities:

$$\dot{b}_i - \dot{b}_j = a_i \int_{b_i}^{b_j} (f - f^*) dx + (a_i - a_j) \int_{b_j}^1 (f - f^*) dx$$

- **First term:** $O(\varepsilon)$ — the integral over a tiny interval. Vanishes as biases approach.
- **Second term:** Proportional to $(a_i - a_j)$. The amplitude difference drives further synchronization.

Positive feedback: Close biases $\rightarrow$ similar dynamics $\rightarrow$ stay close $\rightarrow$ amplitudes equalize $\rightarrow$ biases stay synchronized. This feedback loop **locks** nearby biases together.

Project 4: Merging & Stability

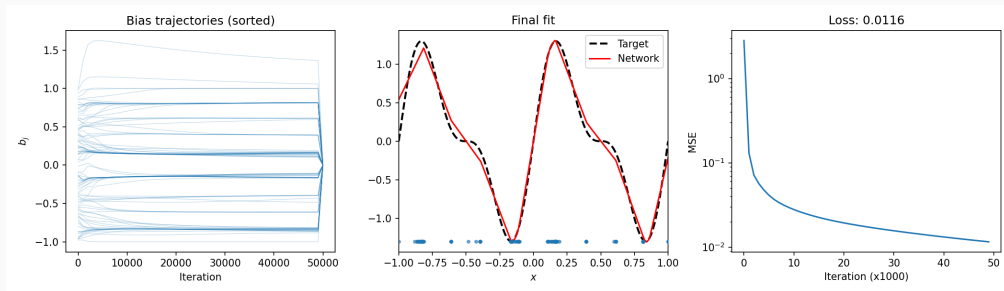
Once merged ($b_i = b_j$): The two neurons are **exactly equivalent** to one neuron.

Stability of the merged state:

- **First order:** The gradient with respect to separating the biases is zero — no incentive to split.
- **Second order:** The Hessian along the separation direction is **positive semidefinite** — splitting increases or maintains the loss.
- **Conclusion:** The merged state is **neutrally stable**. Once biases merge, gradient flow won't separate them.

Overall process: Biases start spread out. Nearby pairs synchronize and merge. The merged groups themselves interact and may merge further. The process continues until only k distinct values remain, where k is determined by the target complexity.

Project 4: Empirical Demonstration



$m = 100$ neurons, target $f^*(x) = \sin(2\pi x) + 0.5 \sin(4\pi x)$ on $[-1, 1]$.

- **Left:** Sorted bias trajectories over 50k iterations. Initial uniform spread $\rightarrow$ tight clusters.
- **Center:** Final fit (red) vs. target (dashed). Dots mark bias locations.
- **Right:** MSE loss on log scale. Smooth decay — no discrete jumps.

Full code in Chapter 11, Section 4.3.

Project 4: What We Learn from the Experiment

Key observations:

1. **Continuous clustering:** Biases approach each other smoothly — there are no discrete collision events. The process is a smooth dynamical flow.
2. **~6–8 clusters:** For a target with 8 curvature changes ($(f^*)''$ sign flips), biases organize into approximately that many groups. This suggests a connection between target complexity and effective architecture size.
3. **Non-uniform cluster sizes:** Some locations attract many biases (high “demand” for a kink), others few. The distribution of cluster sizes may reflect local approximation difficulty.
4. **Smooth loss decay:** Despite massive structural reorganization (100 biases $\rightarrow$ ~8 effective), the loss decreases smoothly.

Project 4: Open Problem 4.1 — Cluster Count

The question: For a given f^* and width m , exactly how many bias clusters emerge?

Conjecture:

$$\lim_{m \rightarrow \infty} C(m, f^*) = \#\{x \in [-1, 1] : (f^*)''(x) = 0 \text{ and changes sign}\}$$

The number of clusters is bounded by the number of **curvature changes** (inflection points) of the target.

Why curvature changes? Each inflection point is where the optimal piecewise linear approximation **must** place a kink. The conjecture says this lower bound is asymptotically tight.

Specific goals:

1. Prove $C(m, f^*) \leq C_{\max}(f^*)$ independent of m .
2. Is convergence to clusters **finite-time** or **asymptotic**?
3. Does the count depend on curvature **amplitude** or only sign pattern?

Project 4: Open Problem 4.2 — Higher Dimensions

The question: Does parameter collapse generalize to $\mathbb{R}^d$ for $d \geq 2$?

General model: $f(x) = \sum_{j=1}^m a_j \sigma(w_j^\top x + b_j), x \in \mathbb{R}^d$.

In 1D, a neuron defines a **point** b_j (the kink location). In $\mathbb{R}^d$, a neuron defines a **hyperplane** $\{x : w_j^\top x + b_j = 0\}$.

Collapse could mean:

1. **Hyperplane collapse:** $(w_i, b_i) \propto (w_j, b_j)$ — same hyperplane.
2. **Directional collapse:** $w_i \propto w_j$ but biases differ — parallel hyperplanes.
3. **Mixed:** Only biases merge, directions remain distinct, or vice versa.

Approach: Start with $d = 2$ and simple targets. Visualize hyperplane configurations.

Project 4: Open Problem 4.3 — Provable Pruning

The question: Can collapsed neurons be **provably** removed without hurting performance?

The pruning procedure: After training, biases cluster into k groups. For each cluster $\mathcal{C}_\ell$ with centroid $\bar{b}_\ell$:

1. Replace all biases in $\mathcal{C}_\ell$ with $\bar{b}_\ell$.
2. Sum their amplitudes: $\bar{a}_\ell = \sum_{j \in \mathcal{C}_\ell} a_j$.
3. Pruned network: $\tilde{f}(x) = \sum_{\ell=1}^k \bar{a}_\ell \sigma(x - \bar{b}_\ell)$.

The guarantee to prove: If intra-cluster distance $\leq \varepsilon$, then:

$$\|\tilde{f} - f\|_\infty \leq \varepsilon \sum_{j=1}^m |a_j|$$

The key challenge: **bound** $\sum |a_j|$ uniformly during training.

Project 4: Broader Connections — Lottery Ticket

Lottery Ticket Hypothesis (Frankle & Carbin, 2019): Within a randomly initialized network, there exists a sparse subnetwork that trains to match the full network's performance.

Connection to bias collapse:

- LTH identifies sparsity **retrospectively**: train, prune, rewind, retrain.
- Bias collapse creates sparsity **prospectively**: training dynamics themselves eliminate redundant neurons.
- **Open**: If we take the clustered network, rewind surviving neurons to initialization, and retrain — does it recover full performance? Are collapsed neurons the “losing tickets”?

Both phenomena point toward a **simplicity bias** in neural network training — the optimizer prefers solutions with lower effective complexity, in both function space and architecture space.

Project 4: Broader Connections — Grokking & Mean-Field

Grokking (Power et al., 2022): Validation accuracy suddenly improves long after training loss saturates.

Bias collapse as a mechanism: The network may first memorize with many neurons (fast, high capacity), then slowly collapse to a sparse solution that generalizes (slow, structural reorganization). The collapse phase corresponds to the grokking period.

Connection to mean-field theory: In the mean-field PDE, bias collapse corresponds to the neuron distribution $\mu(t)$ **concentrating onto a discrete support** — from a diffuse distribution to a sum of Dirac masses. This is a **singularity formation** in the PDE, potentially analyzable with tools from nonlinear PDE theory.

=====

Essential References

1. Neal (1994). *Bayesian Learning for Neural Networks*. PhD thesis, U. Toronto.
2. Williams (1997). *Computing with Infinite Networks*. NeurIPS.
3. Cho & Saul (2009). *Kernel Methods for Deep Learning*. NeurIPS.
4. Jacot, Gabriel, Hongler (2018). *Neural Tangent Kernel: Convergence and Generalization in NN*. NeurIPS.
5. Lee et al. (2019). *Wide Neural Networks of Any Depth Evolve as Linear Models*. NeurIPS.
6. Chizat et al. (2019). *On Lazy Training in Differentiable Programming*. NeurIPS.
7. Chizat & Bach (2018). *On the Global Convergence of Gradient Descent for Over-parameterized Models*. NeurIPS.
8. Mei, Montanari, Nguyen (2018). *A Mean Field View of the Landscape of Two-Layer Networks*. PNAS.
9. Schoenholz et al. (2017). *Deep Information Propagation*. ICLR.
10. Basri et al. (2019). *The Convergence Rate of Neural Networks for Learned Functions of Different Frequencies*. NeurIPS.
11. Bietti & Mairal (2019). *Group Invariance, Stability, and Deformations*. NeurIPS.
12. Xu et al. (2019). *Frequency Principle: Fourier Analysis of Training Dynamics*. ICLR.
13. Matthews et al. (2018). *Gaussian Process Behaviour in Wide Neural Networks*. ICLR.
14. Saxe, McClelland, & Ganguli (2014). *Exact Solutions to the Nonlinear Dynamics of Learning in Deep Linear Networks*. ICLR.
15. Arora, Cohen, Hu, & Luo (2019). *Implicit Regularization in Deep Matrix Factorization*. NeurIPS.
16. Nesterov (1983). *A Method for Solving the Convex Programming Problem with Convergence Rate $O(1/k^2)$* . Doklady AN SSSR